中山大學
SUN YAT-SEN UNIVERSITY

Lecture 13
Recursion



Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

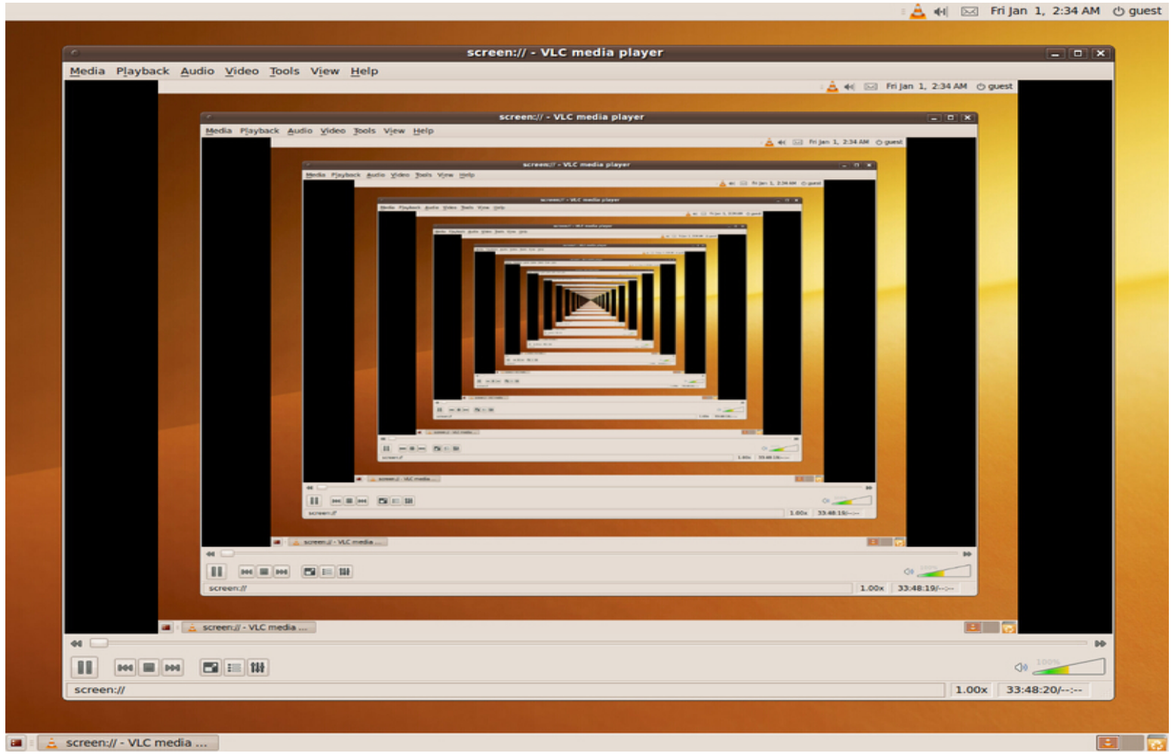
zhangzizhen@gmail.com

Outline

- Introduction to Recursion
- Principles of Recursion
- Tail Recursion

Definition

- An algorithm is **recursive** if it calls itself (directly or indirectly) to do part of its work.
- 递归是一种描述和解决问题的基本方法，用来解决可归纳描述的问题，或说可分解为结构自相似的问题。所谓结构自相似，是指构成问题的部分与问题本身在结构上相似。



Factorials: A Recursive Definition

- The factorial function of a positive integer *n* is

$$n! = n \times (n-1) \times \cdots \times 1$$

- Recursive definition:

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1)! & \text{if } n > 0 \end{cases}$$

Every recursive process consists of two parts:

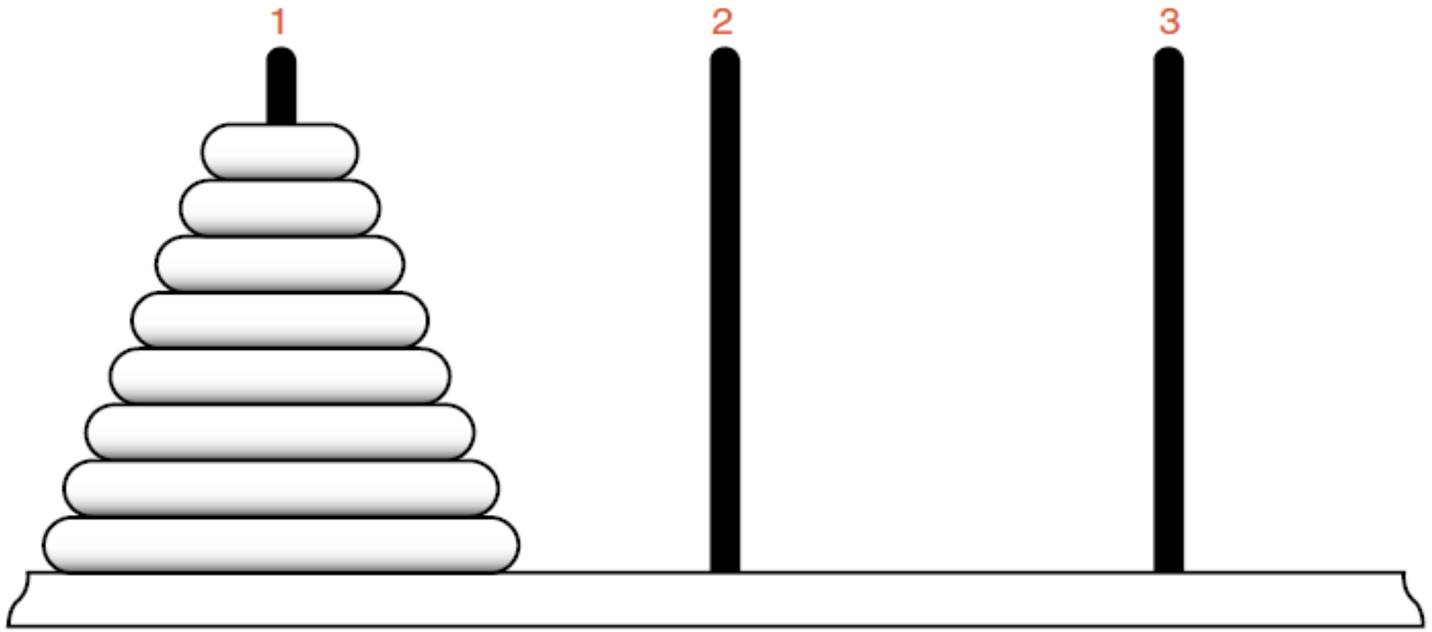
- A smallest, base case that is processed without recursion; and
- A general method that reduces a particular case to one or more of the smaller cases, thereby making progress toward eventually reducing the problem all the way to the base case.

Factorials: A Recursive Definition

```
int factorial(int n)
/* Pre:   n is a nonnegative integer.
   Post:  Return the value of the factorial of n.  */
{
   if (n ==  0)  return 1;
   else          return n * factorial(n — 1);
}
```

- Advantage: the recursive definition and recursive solution of a problem can be both concise and elegant
- Disadvantage: the computational details can require keeping track of many partial computations before the process is complete.

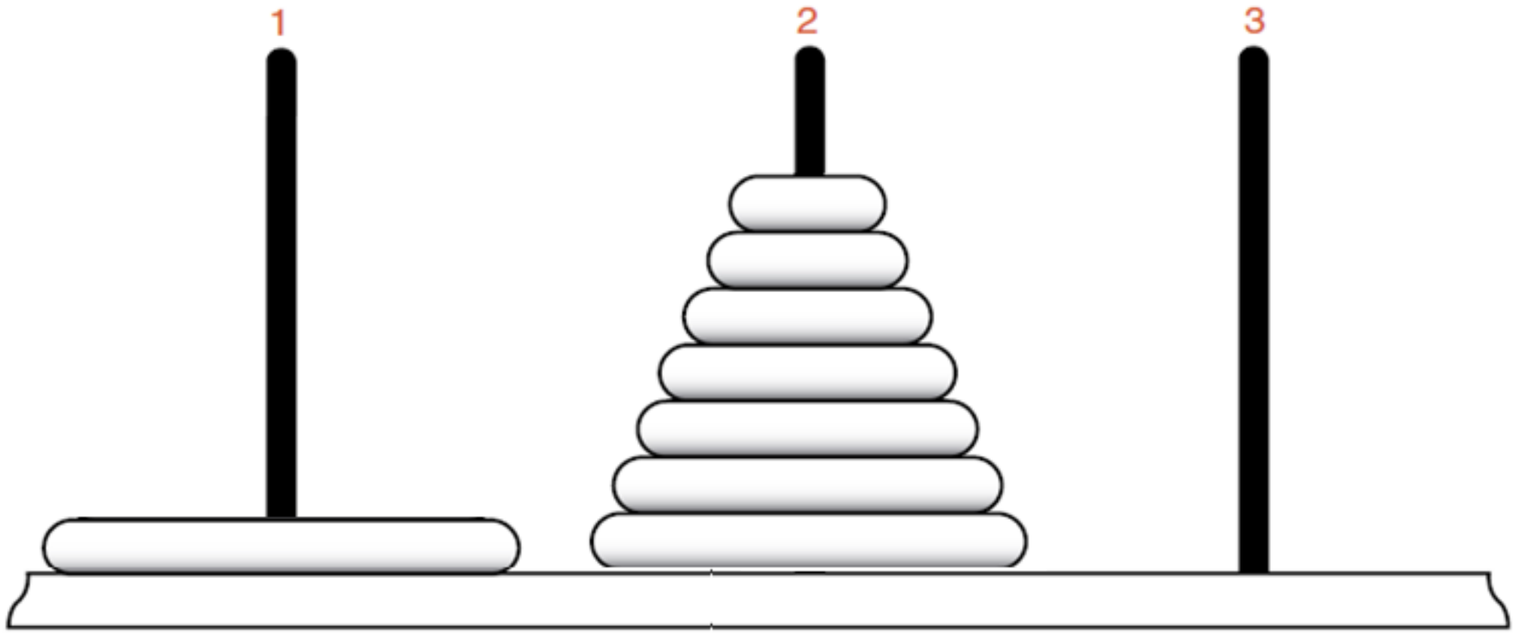
Example: Towers of Hanoi



- **Task**: Move the disks from Tower 1 to Tower 3 using Tower 2 as temporary storage.
- **Rules:**
  - Move only one disk at a time.
  - No larger disk can be on top of a smaller disk.

Example: Towers of Hanoi

- We have to get to this state and then move the bottom disk from Tower 1 to Tower 3.



- Movement of disks in Tower 2 form a similar problem with smaller problem size.

Program for Hanoi

```
const int disks = 64;       //    Make this constant much smaller to run program.
void move(int count, int start, int finish, int temp);


/* Pre:   None.
   Post:  The simulation of the Towers of Hanoi has terminated.  */

main( )
{
  move(disks, 1, 3, 2);
}

void move(int count, int start, int finish, int temp)
{
  if (count > 0) {
    move(count – 1, start, temp, finish);
    cout  ≪ "Move disk " ≪ count ≪ " from " ≪ start
          ≪ " to " ≪ finish ≪ "." ≪ endl;
    move(count – 1, temp, finish, start);
  }
}
```

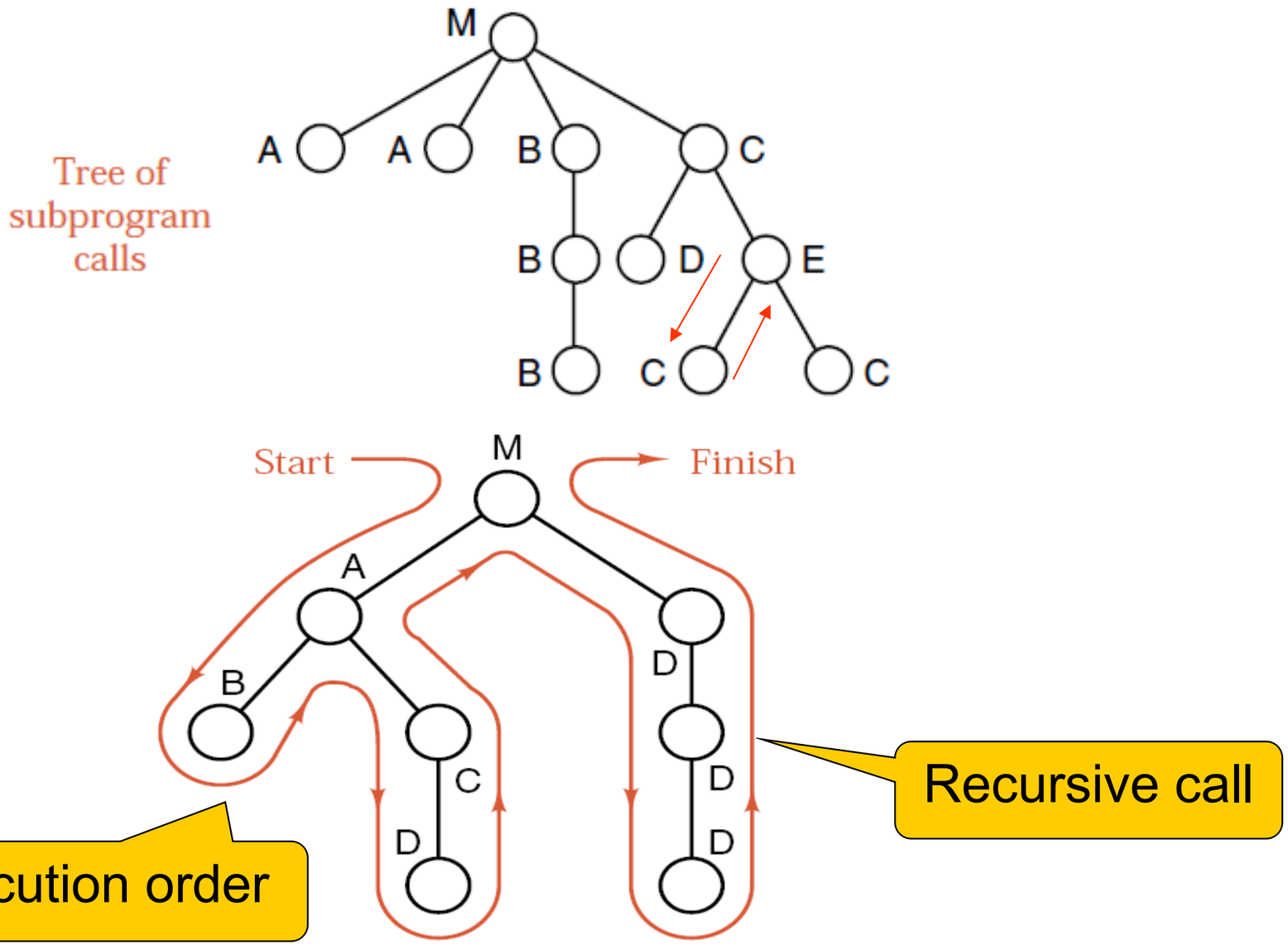
Designing Recursive Algorithms

- Find the key steps
- Find a stopping rule
- Outline your algorithm
- Check termination
- Draw a recursion tree

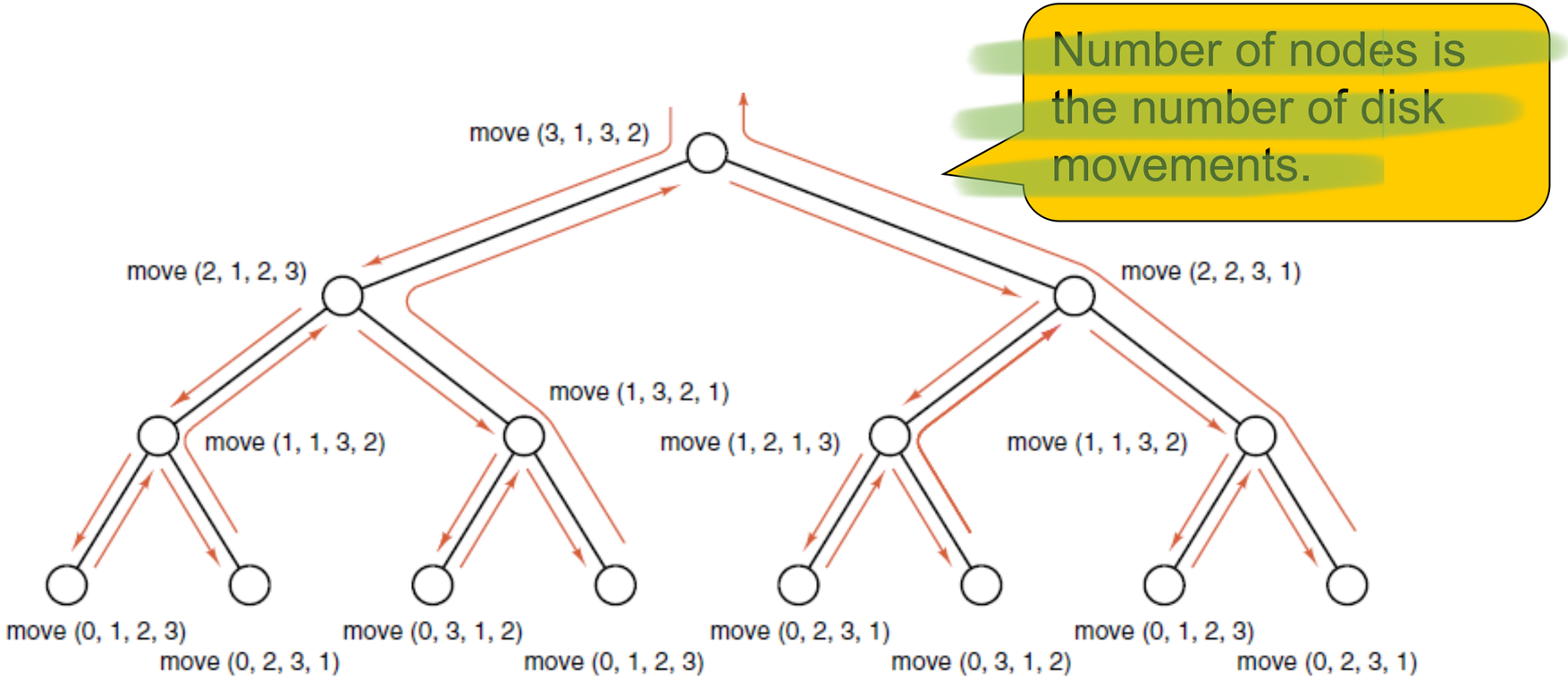
Tree of Function Calls (函数调用树)

- Tree of function calls is a tool to analyzing the process of function calls.
- A node denotes a function call with the function name;
- A function may invokes other functions, which are denoted as children;
- Different recursive calls appear as different vertices with the same name;
- Recursion tree: the recursive part of the tree of function calls.

Tree of Function Calls and Execution Order



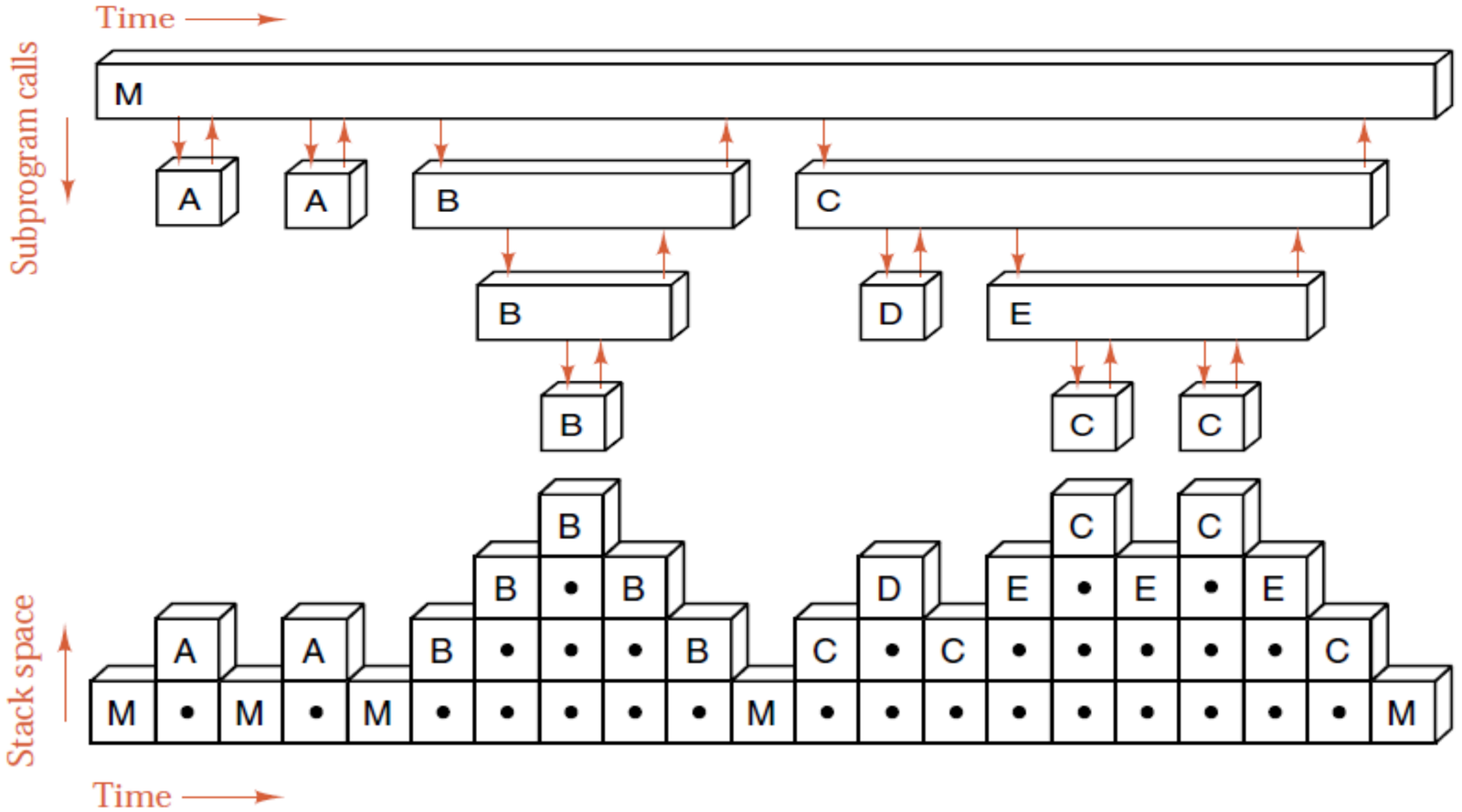
Recursion Tree for Hanoi Tower



- From the recursion tree above we can get the number of moves (the number of inner vertices) to solve Hanoi for 64 disks: $1+2+2^2+ \ldots +2^{63} = 2^{64} - 1 \approx 1.6 \times 10^{19}$

Stack Frames (栈帧)

- **栈帧(stack frame) / 活动记录(activation record)**——为每个调用函数分配的临时存储区域，包括返回地址、调用参数、局部变量等。
- **函数嵌套调用**——函数按“后进先出”次序；执行函数所使用的临时存储区域也具有“后进先出”的特性；于是调用记录必须用“**堆栈**”保存。



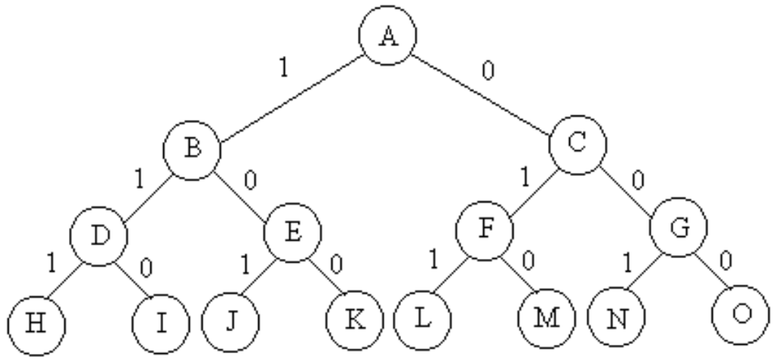
Stack Frames

- When a program calls a subprogram, the system:
  - Store returning location and parameters;
  - Allocate storage for all the local variables.
  - Then the control is passed to the subprogram.
  - (i.e., The activation record is pushed into the stack)
- When the subprogram is finished, the system:
  - Return the result of the subprogram;
  - Release the storage allocated for the subprogram;
  - The control is passed to the caller and execution starts at the returning address.
  - (i.e., The activation record for the function is popped out)
- A function may call itself. There is no difference between this recursive call and other function calls.

When to Use Recursion

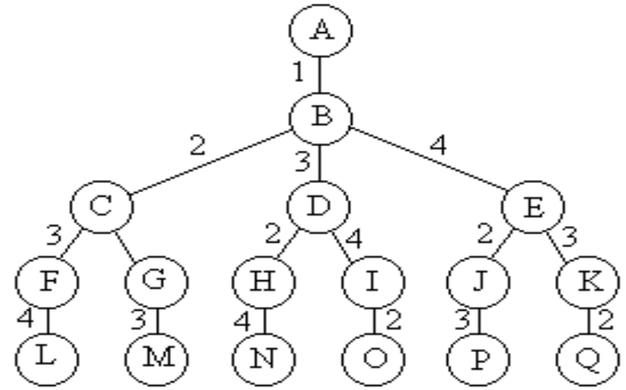
- If time and space is not an issue, then recursive version is much easier to understand and preferred.
- If duplicate tasks are involved, then data structure other than stacks will be appropriate.
- Recursion can always be replaced by iteration and stacks. But no point to do it if not necessary.

Generating Subsets and Permutations



Enumerating all subsets take $O(2^n)$

```
void backtrack(int t)
{
  if (t>n) output(x);
  else
      for (int i=0;i<=1;i++) {
        x[t]=i;
        if (legal(t)) backtrack(t+1);
      }
}
```



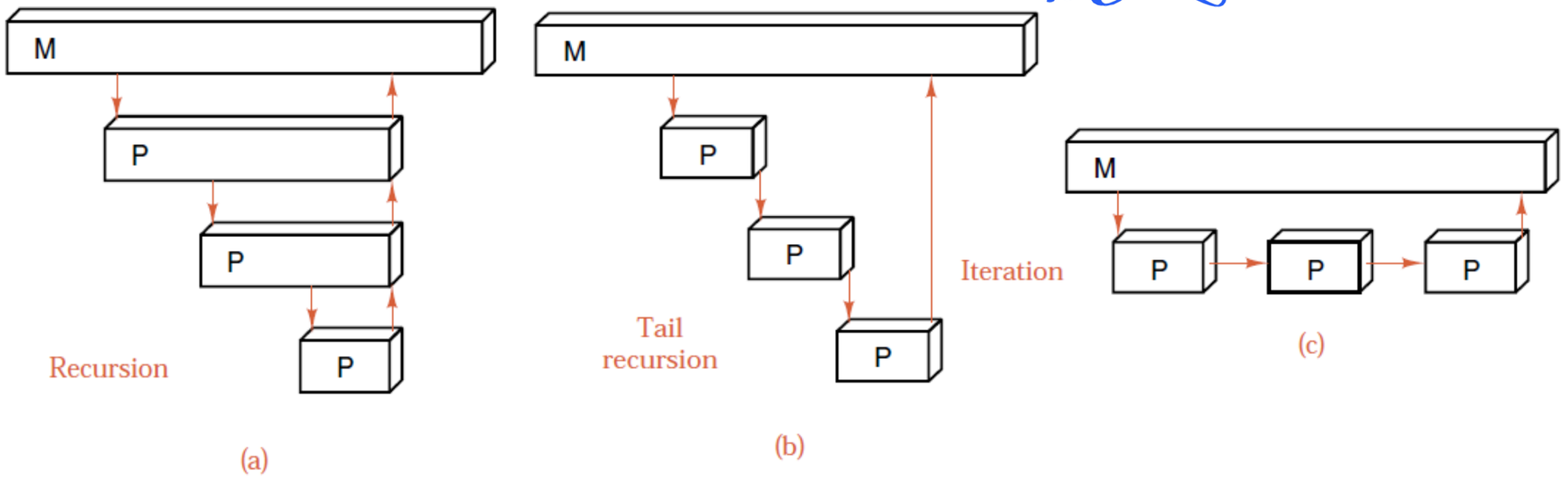
Enumerating all permutations take O(n!)

```
void backtrack(int t)
{
  if (t>n) output(x);
    else
      for (int i=t;i<=n;i++) {
        swap(x[t], x[i]);
        if (legal(t)) backtrack(t+1);
        swap(x[t], x[i]);
      }
}
```

Tail Recursion

DEFINITION ***Tail recursion*** occurs when the last-executed statement of a function is a recursive call to itself.

If the last-executed statement of a function is a recursive call to the function itself, then this call can be eliminated by reassigning the calling parameters to the values specified in the recursive call, and then repeating the whole function.



Hanoi Without Tail Recursion

```
void move(int count, int start, int finish, int temp)
{
   if (count > 0) {
      move(count — 1, start, temp, finish);
      cout ≪ "Move disk " ≪ count ≪ " from " ≪ start
           ≪ " to " ≪ finish ≪ "." ≪ endl;
      move(count — 1, temp, finish, start);
   }
}
```

tail recursion

```
void move(int count, int start, int finish, int temp)
/* move: iterative version
   Pre:  Disk count is a valid disk to be moved.
   Post: Moves count disks from start to finish using temp for temporary storage.  */
{
   int swap;                                   //   temporary storage to swap towers
   while (count > 0) {                         //   Replace the if statement with a loop.
      move(count — 1, start, temp, finish);    //   first recursive call
      cout ≪ "Move disk " ≪ count ≪ " from " ≪ start
           ≪ " to " ≪ finish ≪ "." ≪ endl;
      count——;                                 //   Change parameters to mimic the second recursive call.
      swap = start;
      start = temp;
      temp = swap;
   }
}
```

Binary Search With/Without Tail Recursion

```
Error_code recursive_binary_2(const Ordered_list &the_list,
                const Key &target, int bottom, int top, int &position)
{
   Record data;
   if (bottom <= top) {
      int mid = (bottom + top)/2;
      the_list.retrieve(mid, data);
      if (data ==  target) {
         position = mid;
         return success;
      }
      else if (data < target)
         return recursive_binary_2(the_list, target,
                                   mid + 1, top, position);
      else
         return recursive_binary_2(the_list, target,
                                   bottom, mid - 1, position);
   }
   else return not_present;
}
```

tail recursion

tail recursion

```
Error_code binary_search_2(const Ordered_list &the_list,
                           const Key &target, int &position)
{
   Record data;
   int bottom = 0, top = the_list.size( ) - 1;
   while (bottom <= top) {
      position = (bottom + top)/2;
      the_list.retrieve(position, data);
      if (data ==  target) return success;
      if (data < target) bottom = position + 1;
      else top = position - 1;
   }
   return not_present;
}
```

Quicksort With/Without Tail Recursion

```
QUICKSORT(A, p, r)
1  if p < r
2       then q ← PARTITION(A, p, r)
3            QUICKSORT(A, p, q − 1)
4            QUICKSORT(A, q + 1, r)
```

tail recursion

```
QUICKSORT'(A, p, r)
1  while p < r
2       do ▸ Partition and sort left subarray.
3          q ← PARTITION(A, p, r)
4          QUICKSORT'(A, p, q - 1)
5          p ← q + 1
```

Thank you!

